Upande Dev Tools Portal — System Reference

A self-contained technical description of this application's data model, permission model, business rules, and computed metrics — written for a language model reading it with no access to the source code, so it can reason about the design and suggest improvements on its own.

How to use this document

You are being asked to review the design of a piece of internal software, based only on what follows. Every field name, role name, threshold, and formula below is taken directly from the running implementation and is current as of the time this document was produced — treat it as ground truth, not paraphrase. Known limitations are called out explicitly in shaded boxes; where something looks like an odd choice and no rationale is given nearby, assume it has not been examined critically. Please ground any suggestion in something stated here, and say so plainly if a suggestion rests on an assumption this document does not confirm.

At the end of this document is a specific set of questions this review is meant to answer. Read the whole thing before answering them — the sections build on each other.

1. What this system is

upande_dev_tools is a custom app running inside a Frappe/ERPNext instance (Python + MariaDB, server-rendered pages) belonging to Kaitet / Karen Roses, a rose-growing business. It is not the business's core ERP — it is an internal tool bolted onto that same instance for managing the software work happening on top of it: feature and bug requests, task tracking, deployment coordination, and reporting on all of it. It has one deployment, one set of users (the same user records as the rest of the ERP), and no public-facing surface.

Frontend: a single shared page shell (a left-hand navigation rail, rendered server-side via Jinja) wraps each page's content. Each page is its own hand-written JavaScript class using jQuery and direct DOM string templates — there is no frontend framework (no React/Vue) and no charting library. Every chart on every page (bar, donut, line/burndown) is hand-rolled as inline SVG or CSS width/height percentages.

Backend: plain Python functions decorated `@frappe.whitelist()`, called from the frontend via `frappe.xcall(...)`. Permission checks are hand-written at the top of each function (`set(frappe.get_roles()) & {ROLE_SET}`) — this app does not rely on Frappe's declarative per-doctype permission rules for its own logic (though those still apply underneath, since every write ultimately goes through Frappe's document layer).

2. Roles and the page-access model

Three custom roles are used throughout: **Dev Team**, **Projects Manager**, and Frappe's built-in **System Manager** (included everywhere as "or an admin"). Anyone authenticated who holds none of these is referred to below as a **plain user**. There is no "Requester" role — being a plain user is itself the requester tier.

Page visibility is **not** Frappe's standard permission system. It runs through a custom doctype, `Dev Portal Page` (route, title, icon, nav_group, sort_order, a child table of allowed roles, and a require_all_roles flag: unchecked = any listed role qualifies, checked = every listed role is required). On every request, `portal.py`'s `enforce_page_access()` looks the route up in this table and silently redirects to the user's own home route if they don't qualify — there is no error page. `get_nav_items()` reads the same table to build the left-hand nav, so a page's visibility and its presence in the nav are the same check.

Full page registry (13 pages), as currently configured:

Nav group	Title	Route	Roles (ANY unless noted)
Admin	Settings	/dev-portal-settings	Dev Team AND System Manager
Admin	Master Data	/master-data	Dev Team, Projects Manager
Developer	Dashboard	/dev-dashboard	Dev Team
Developer	Hooks Explorer	/hooks-explorer	Dev Team
Developer	Code Editor	/code-editor	Dev Team
Developer	My Backlog	/my-backlog	Dev Team
Developer	Backlog Board	/backlog-board	Dev Team, Projects Manager
Developer	Code Snapshots	/code-snapshots	Dev Team
Developer	Activity Log	/activity-log	Dev Team
Management	Dashboard (Portfolio)	/pm-dashboard	Projects Manager
Management	Review Queue	/review-queue	Projects Manager
Management	Deployments	/deployments	Dev Team, Projects Manager, System Manager
Management	Requests	/requests-portal	All (every authenticated user)

A signed-in user's home route (where a bare login lands them) is: Dev Team → /dev-dashboard; Projects Manager → /pm-dashboard (Dev Team checked first, so a user with both roles lands on the dev dashboard); anyone else → /requests-portal.

API-level permission gates (separate from, and not derived from, the page registry above)

Constant	Module	Roles	Gates
REVIEWER_ROLES	requests.py	Dev Team, Projects Manager, System Manager	triaging/accepting requests; who may set priority/module/tags
BOARD_ROLES	board.py	Dev Team, Projects Manager, System Manager	editing the shared backlog board directly
MANAGER_ROLES	portfolio.py	Projects Manager, System Manager	the entire Portfolio Dashboard API
DEPLOYER_ROLES	deployments.py	Dev Team, System Manager	creating/listing deployment apps & instances, the deploy queue
DEPLOYMENT_VIEWER_ROLES	deployments.py	Dev Team, Projects Manager, System Manager	reading the full recent-deployments list
DEPLOYMENT_APPROVER_ROLES	deployments.py	Projects Manager, System Manager	approving/rejecting a peak-hours-flagged deployment
MASTER_DATA_ROLES	master_data.py	Dev Team, Projects Manager	editing shared reference lists

Constant	Module	Roles	Gates
DEV_TOOLS_ROLES	code_editor.py / hooks_explorer.py / activity_log.py / code_snapshots.py / dashboard.py	Dev Team	each dev-only utility, independently

Noted risk: page-level access (Dev Portal Page records) and API-level access (the role-set constants above) are two independently maintained sources of truth with no cross-check between them. It is possible for a page to be reachable by a role that then fails on every API call it makes, or for an API method to be reachable by a role that has no page surfacing it. Nothing currently verifies the two stay in sync.

3. Data model

3.1 Doctypes this app owns

Request — the single intake point for anything anyone wants built or fixed.

Field	Type	Notes
title	Data	
request_type	Link → Request Type	master data
priority	Link → Priority Level	server-side dropped to null for non-reviewers, see §5
workflow_state	Select	see §4 workflow
description	Text Editor	
product_area	Link → Product Area	master data; same non-reviewer drop as priority
project	Link → Project	set post-insert by db.set_value, not on create — see §5
linked_task	Link → Task	populated when scheduled
raised_by_employee	Link → Employee	
requested_assignee	Link → User	
source	Select	Desk / Web Portal / Mobile App
resolution_notes	Small Text	

Deployment Request — one row per deployment attempt.

Field	Type	Notes
app	Link → Deployment App	
instance	Link → Deployment Instance	
branch / commit_hash	Data	
workflow_state	Select	Requested / In Progress / Deployed / Failed
linked_request	Link → Request	optional
requested_by_user / requested_by_employee	Link	
deployed_by_user	Link → User	
requires_approval	Check	set once, at insert — see §6
approval_status	Select	"" / Pending / Approved / Rejected
approved_by / approved_on / approval_note	Link / Datetime / Small Text	
errors / fix_notes	Text	

Dev Portal Settings — a singleton (one row, no list view): peak_hours_enabled (Check), peak_start_time (Time), peak_end_time (Time). **Dev Portal Page** — described in §2. **Deployment App** / **Deployment Instance** — simple named master data (app default branch; instance environment type). Several small master-data doctypes back

Request's product_area / request_type / priority link fields and are editable from the Master Data admin page.

3.2 Core doctypes this app extends rather than owns

Task (ERPNext core) carries two custom fields this app added: `custom_module` (free text/link, used for the module breakdowns throughout the dashboards) and `custom_request` (Link → Request, set when a task originated from an approved request; left blank for a task created directly on the board). Every dashboard metric that reads "open/overdue/delivered" work is reading Task's native `status`, `exp_end_date` (due date) and `completed_on` (set automatically by ERPNext when status becomes Completed) — none of these are custom fields.

Project carries `custom_project_scope` (Internal / External) and is tagged with a `project_type` value of "Dev Tools" to mark it as in-scope for the Portfolio Dashboard; a project without that tag is invisible to every portfolio metric even if it has tasks. **Issue** (ERPNext core) is used only for its "On Hold" status, folded into the portfolio's risk list as blocked work.

4. Workflows

Request Review (doctype: Request)

From	Action	To	Who
Under Review	Approve	Approved	Projects Manager
Under Review	Reject	Rejected	Projects Manager
Under Review	Defer	Deferred	Projects Manager
Under Review	Promote Note	Scheduled	Dev Team
Deferred	Reopen	Under Review	Projects Manager
Approved	Schedule	Scheduled	Projects Manager
Scheduled	Start Work	In Progress	Dev Team
In Progress	Complete	Completed	Dev Team
Under Review / Approved / Deferred / Scheduled / In Progress	Withdraw	Withdrawn	anyone (the raiser)
Completed	Confirm Complete	Closed	anyone (the raiser)

Scheduling a Request (Approved → Scheduled) is what creates the linked Task. A Note-type request can be self-promoted straight to Scheduled by Dev Team without Projects Manager approval ("Promote Note") — the only transition in this workflow a requester's own role can drive without a manager step, and it is restricted to Dev Team, not open to plain users raising notes.

Deployment Review (doctype: Deployment Request)

From	Action	To	Who
Requested	Start Deployment	In Progress	Dev Team
In Progress	Mark Deployed	Deployed	Dev Team
In Progress	Mark Failed	Failed	Dev Team
Failed	Retry	In Progress	Dev Team

Projects Manager never drives this workflow directly — their only lever is the separate approval gate described next, which blocks the Requested→In Progress transition rather than being a state in the workflow itself.

5. The Request → Task pipeline, and what a plain user cannot set

`api.board.create_task()` is the entry point for adding work to the shared backlog board. If the caller holds a BOARD_ROLES role, it creates a Task directly. If not, it silently redirects into `api.requests.create_request()` instead and returns a Request — **the caller is never told the object type changed**; the frontend already only offers a "request" framing to non-privileged users, so this is a server-side backstop against someone calling the API directly, not a UX moment.

That bridge deliberately does not forward a `project` value, even if the caller supplied one — discovered live during development that a plain user calling with a project they don't have Frappe-level read access to throws inside create_request's own permission check. The project is attached later by whoever accepts the request instead.

Independently, `create_request()` itself silently drops `product_area` and `priority` to null for any caller outside REVIEWER_ROLES, regardless of what value the client sent — the frontend hides those form fields from non-reviewers already, but the server does not trust that and re-enforces it. A reviewer taking ownership of a request sets these fields later, during triage.

6. The peak-hours deployment gate

`is_peak_hours(at=None)` reads the singleton Dev Portal Settings row and returns whether a given moment (default: now) falls inside the configured window. If peak_hours_enabled is off, or either time is unset, it always returns False. If start == end, it always returns True (a zero-width window is read as "always," not "never"). If start < end it is a same-day window; if start > end it correctly wraps past midnight (e.g. 20:00–06:00).

`DeploymentRequest.before_insert()` calls this once, at creation, and if true sets requires_approval=1 and approval_status="Pending" permanently — **a later change to the peak-hours window, or to the setting itself, never retroactively flags or unflags an already-created deployment.**

Enforcement is duplicated on purpose. `api.deployments.update_deployment_status()` throws a friendly, specific error before attempting the "Start Deployment" action if requires_approval is set and approval_status isn't "Approved." But Frappe's desk UI can drive the same workflow directly via `apply_workflow()`, bypassing that whitelisted method entirely for anyone with desk access to the doctype — so the real backstop is `DeploymentRequest.validate()`, which throws frappe.PermissionError whenever workflow_state is being changed to "In Progress" while requires_approval is true and approval_status isn't "Approved," regardless of which code path drove the change. This was verified with a test that calls apply_workflow directly, bypassing the API.

Approval itself (`approve_deployment_request` / `reject_deployment_request`) is gated to DEPLOYMENT_APPROVER_ROLES (Projects Manager, System Manager) and throws if the target deployment was never flagged in the first place — approval is not a generic status field, it is meaningless outside the peak-hours context.

7. Portfolio Dashboard — derived metrics (api/portfolio.py)

Every function below operates on a **scope**: all Tasks/Requests/Issues attached to a Project whose project_type is "Dev Tools" (constant DEV_TOOLS_PROJECT_TYPE), optionally narrowed further to custom_project_scope = Internal or External. Anything with no Project at all is excluded outright; its count is surfaced separately as period.excluded so the dashboard states plainly how much work it isn't counting, rather than silently under-representing the org's total load. get_portfolio(days, scope) is gated to MANAGER_ROLES and is the single entry point; every metric below is one key of its return dict.

7.1 Headline KPIs (kpis)

key	What it is	Formula / window
delivered	Tasks completed in the period	count(status=Completed, completed_on in [start,end])
cycle	Median cycle time, days	median(completed_on – creation) over delivered tasks
on_time	% finished by their own due date	of delivered tasks that had exp_end_date set, % with completed_on ≤ exp_end_date; None if none had a due date
net	Backlog change, signed	raised – delivered in the period; negative = shrinking
waiting	Requests with no decision yet	count(workflow_state in ["", "Under Review"]); note also gives the oldest one's age in days
risk	"At risk now"	overdue-open-task count + On-Hold Issue count

Each KPI also carries a period-over-period delta and a good/bad direction (whether "better" means up or down is hardcoded per KPI, e.g. rising cycle time is bad, rising delivered is good).

7.2 Ageing bands (ageing) — how old currently-open work is, by creation date

Fixed six bands, first match wins: Under a week (<7d) · One to two weeks (<14d) · Two to four weeks (<30d) · One to two months (<60d) · Two to six months (<180d) · Over six months (everything else). Not configurable.

7.3 "Over capacity" per person (part of people)

Computed per assignee over currently-open tasks only. Two independent, either-triggers conditions, both requiring at least one other real assignee to compare against (avg_open = mean open count across everyone with any open work, excluding the "Unassigned" bucket):

Reason	Condition	Intuition
volume	open > 2 × avg_open AND overdue > 0	carrying a much bigger pile than peers, and it's not all on-time
lateness	open ≥ 5 AND (overdue / open) ≥ 0.3	a third or more of their own queue is already late, regardless of its size

A person can trigger both ("both"); the flag and its reason are exposed per person so the UI can say *why*, not just flag a name.

7.4 Modules, requests, due-soon, stalled (all in the same request)

- **modules** — top 10 modules by (open + late + delivered-this-period), each broken into those three counts; "late" here means open with exp_end_date before today, distinct from the ageing bands (which key off creation date, not due date).
- **requests** — total / accepted (Approved+Scheduled+In Progress+Completed) / rejected / deferred / waiting, plus the longest current wait in days.
- **due_soon** — open tasks with exp_end_date in [today, today+7], capped to 10 rows, plus the true total count separately (so the UI can say "16 due, showing 10").
- **stalled** — open tasks not modified in 14+ days, oldest first, capped to 8.
- **risks** — a merged, single list of overdue tasks + On-Hold issues + undecided requests, sorted worst-first by age in days regardless of type, capped to 10.

7.5 Backlog forecast (burndown) — the most involved computation in the app

Two distinct time series, joined by one calibration number:

History (past 8 weeks, one point per week boundary):

- `open` — reconstructed backlog size as of that date: count(status≠Cancelled, created ≤ date) – count(status=Completed, completed_on ≤ date). Cancelled tasks are excluded from the reconstruction entirely (no cancelled-on timestamp exists to place them on the timeline) — a task cancelled last week reads as still-open at every earlier checkpoint.
- `estimated` — how many tasks (any current status) had exp_end_date inside that specific week.
- `actual` — how many tasks were actually completed inside that same week.

hit_rate = mean(actual/estimated) across only the weeks where estimated > 0, **defaulting to 1.0 (100%) if no week in the whole 8-week window ever had anything due.**

Future (next 4 weeks):

- `estimated` — currently-open tasks whose exp_end_date falls in that future week (what's already on the books, not a guess).
- `forecast` = round(estimated × hit_rate) — what's realistically expected to actually close, discounted by how the team has actually performed against due dates.

- the backlog-size projection for that week = max(0, previous level + avg_raised − forecast), where avg_raised is backed out arithmetically from the history series itself (open[i] + actual[i] − open[i−1]) rather than queried separately.

> **Noted risk:** hit_rate's fallback of 1.0 when there is no historical due-date data is an *optimistic* default ("assume everything gets done on time"), which will make a forecast for a team or project with no completion history look far rosier than a conservative default (e.g. 0, or the on_time KPI) would. This has not been reconsidered since it was first written.
>
> **Also noted:** a date-boundary bug was found and fixed during development — comparing a full datetime column against a bare date string as an upper bound ("<= as_of") silently excludes anything created or completed later that same day, because the string is treated as midnight. The fix (exclusive upper bound at the start of the next day) is applied throughout _burndown, but the same class of bug is worth checking for anywhere else in the codebase that filters a datetime field by a bare date.

8. My Backlog — personal metrics (api/requests.py)

`get_my_backlog(user=None, project=None)` defaults to the caller; a caller may only pass a different user if they hold a REVIEWER_ROLES role. It returns: every open task assigned to that user (via the native _assign field), sorted by due date with undated tasks sorted last (not first, so a real deadline always outranks "someday"); that user's upcoming meetings within 1 day; every Request that user personally raised, in *any* workflow_state (distinct from the public Requests list, which shows everyone's); and a completion-trend object.

Completion trend: this-week vs previous-week completed-task counts for that user, computed purely from Task.completed_on (native to ERPNext, set automatically) — week boundaries are rolling 7-day windows ending today, not calendar weeks.

The frontend additionally derives, client-side from the same task list: a 7-day-ahead due-date count per day ("the week ahead"), a 4-band age histogram (creation-date based, same shape as §7.2 but only 4 bands), and a status/module breakdown chart — none of these are separate API calls.

9. Interface conventions

- Light theme throughout (no dark mode); design tokens are CSS custom properties in one shared stylesheet (dev-portal.css) used by every page.
- No charting library anywhere in the app. Bars are literal width/height percentages; donuts are inline SVG <circle> elements with stroke-dasharray segments; the burndown/forecast chart is an inline SVG polyline with a solid "measured" segment and a dashed "projected" segment sharing one boundary point.
- Every list-heavy page follows the same shell: a toolbar (title, refresh, filters) then either a plain content stage or a row of tabs above it. Tabs were added incrementally to the two data-dense pages (Portfolio Dashboard: Pulse/Delivery/People/Aging & Risk/Modules/Forecast; My Backlog: Today/Tasks/Requests/Progress) once a single continuous scroll became too long; this is treated as the pattern for any future page with several unrelated groups of content, not something special-cased to those two.
- A systemic spacing bug was found and fixed: the shared card component carries no margin of its own, so any two cards rendered as bare siblings (not inside a grid wrapper that supplies its own gap) sat flush against each other — rounded corner against square edge, shadows touching — which read as "overlapping" even though nothing was actually stacked. Fixed once, generically, for every page (any card or grid landing directly in a page's content area gets a consistent margin), rather than per-component.
- Status/priority/kind chips use a small fixed palette of tone classes (e.g. st-triage, st-in-progress, st-blocked, st-done; pr-urgent, pr-high, pr-medium, pr-low) shared across every page so the same concept always reads the same color everywhere it appears.

10. What this review should answer

Based only on what's described above, please give an independent assessment covering:

- **Correctness / data-integrity risks** — places where the stated logic could silently produce a wrong number or wrong permission decision, beyond the two already flagged in shaded boxes.
- **Gaps between stated intent and actual rule** — anywhere a rule as described doesn't fully achieve the reasoning given for it.
- **Missing safeguards or edge cases** — states, role combinations, or timing windows this document doesn't show a rule for.
- **Analytical/methodology critique** — are the KPI definitions, the over-capacity thresholds (§7.3), and the forecast's hit-rate calibration (§7.5) sound statistically, and would you define any of them differently given the same underlying data?
- **Information-architecture / UX issues** — implied by the role table and page list in §2: is anything mis-scoped, duplicated, or missing a home?
- **Anything you flag as an assumption** — be explicit when you're inferring beyond what's stated here rather than reading it directly off a fact in this document.